Does One LoRA Run Beat Prompt Engineering? A Controlled Study on GSM8K with Qwen2.5-1.5B-Instruct

Mete Han Yılmaz
Student No. 26587026
YZM5109 Large Language Models, Yıldız Technical University

***Abstract*—We test whether a single LoRA fine-tuning run outperforms prompt engineering on grade-school math (GSM8K) with Qwen2.5-1.5B-Instruct. Four conditions are evaluated on all 1,319 test problems with identical greedy decoding: zero-shot direct prompting (C1), zero-shot chain-of-thought (C2), Program-Aided Language modelling (PAL, C3), and a LoRA fine-tune evaluated with the exact C1 prompt (C4). Zero-shot CoT is best (75.4%), 4.5 points above direct prompting (70.9%, McNemar $p{=}2.7{\times}10^{-4}$). The fine-tuned model is *worse* than the untouched model (59.6%, -11.3 points, $p{<}10^{-13}$), despite training only 1.18% of the parameters for 9 minutes. Per-item analysis explains why: fine-tuning on the human-written GSM8K solutions shortened the model's reasoning by 56%, and the accuracy gap to the original model grows from 7 to 21 points as problems get longer, while per-equation arithmetic accuracy is unchanged. The model's own reasoning style was better supervision than the dataset's references.**

I. Introduction

A pretrained LLM can be adapted to a task by changing its inputs (prompting) or its weights (fine-tuning). Prompting costs nothing at training time but can lengthen every answer; low-rank adaptation (LoRA) [1] trains a small fraction of the parameters but needs labelled data and GPU time. For small *instruction-tuned* models the trade-off is not obvious: such models have already been tuned heavily on reasoning data, so a short task-specific fine-tune may help, do nothing, or hurt. We answer the assignment's question, *does one LoRA fine-tuning run outperform prompting on the original model?*, in a controlled setting where only the adaptation technique varies, and we use the per-item predictions to explain the outcome rather than only report it.

II. Experimental Setup

A. Model and data

Model. Qwen2.5-1.5B-Instruct [2], a 1.56B-parameter decoder-only transformer with grouped-query attention, the largest recommended size that trains comfortably on one GPU. **Dataset.** GSM8K [3]: 7,473 training and 1,319 test problems, each with a human-written step-by-step solution and one numeric answer. A numeric answer allows unambiguous exact-match scoring, avoiding the judge noise of free-text tasks. All 1,319 test problems are used.

B. Conditions

Every condition uses the same system prompt ("You are a helpful assistant.") and the same answer-format instruction (*End your response with "The answer is: <number>"*), so output format is held fixed.

- **C1 Zero-shot direct:** "Solve the following math problem. Question: ..." plus the format line.
- **C2 Zero-shot CoT** [5], [4]: C1 plus an explicit instruction to reason step by step, write out every intermediate calculation and check the result.
- **C3 PAL** [6]: the model writes a Python function `solution()`; we run it in an isolated subprocess (5 s timeout) and score its return value, delegating arithmetic to the interpreter.
- **C4 LoRA:** the original model fine-tuned once with LoRA, evaluated with the *identical* C1 prompt. Prompting techniques are not re-applied to C4.

C. Fine-tuning (C4)

Training pairs are (C1 prompt, reference solution): the GSM8K rationale with its calculator annotations `<<...>>` removed, followed by "The answer is: N". The loss is computed on response tokens only. LoRA adapters ($r{=}16$, $\alpha{=}32$, dropout 0.05) are attached to all attention and MLP projections (`q,k,v,o,gate,up,down`), giving 18.46M trainable parameters (1.18%). We use AdamW, learning rate $2{\times}10^{-4}$ with 5% warm-up and cosine decay, effective batch 16 (4×4 gradient accumulation), 300 optimiser steps (4,800 examples, 0.64 epochs, 0.93M tokens), maximum length 512, gradient checkpointing and bf16 mixed precision on one NVIDIA A100-40GB.

D. Decoding and evaluation

Decoding is identical for every condition: greedy (`do_sample=False`), at most 512 new tokens, no repetition penalty. The predicted answer is the number after "The answer is", falling back to `\boxed{}` and then to the last number in the output; for PAL it is the program's result. A prediction is correct if it equals the reference within 10^{-4}. We report accuracy with 95% bootstrap CIs (10,000 resamples) and compare every pair of conditions with an exact two-sided McNemar test [9], the appropriate test for paired binary outcomes on the same items.

TABLE I
GSM8K TEST ACCURACY, n=1,319, IDENTICAL GREEDY DECODING. CI: 95% BOOTSTRAP. TOK.: MEAN GENERATED TOKENS. TRUNC.: % OF ANSWERS THAT HIT THE 512-TOKEN LIMIT. NO ANS.: % WITH NO EXTRACTABLE NUMBER OR A FAILED PAL PROGRAM.

Condition	Acc. (%)	95% CI	Tok.	Trunc.	No ans.
C1 Direct	70.9	[68.4, 73.3]	217	1.0	0.0
C2 CoT	**75.4**	[73.0, 77.7]	278	2.6	0.0
C3 PAL	59.7	[57.0, 62.4]	110	0.1	7.4
C4 LoRA	59.6	[56.9, 62.2]	95	0.6	0.0

TABLE II
PAIRED EXACT MCNEMAR TESTS FOR "X VS. Y". b: ITEMS ONLY X SOLVES; c: ITEMS ONLY Y SOLVES.

Comparison	ΔAcc. (pts)	b	c	p
C2 vs. C1	+4.5	157	98	2.7×10^{-4}
C3 vs. C1	−11.2	127	275	1.2×10^{-13}
C4 vs. C1	−11.3	114	263	1.1×10^{-14}
C4 vs. C2	−15.8	89	297	2.9×10^{-27}
C4 vs. C3	−0.1	200	201	1.0

III. RESULTS

Table I and Fig. 1 show a clear ranking: C2 > C1 > C3 ≈ C4. Every difference is significant except C3 vs. C4 (Table II). Explicit CoT gains 4.5 points over direct prompting at a cost of 28% more tokens. The fine-tuned model loses 11.3 points against the very model it was trained from. Fine-tuning did not fail to run: the training loss fell from about 0.6 to a plateau of about 0.41 within roughly 50 steps (Fig. 2).

IV. ANALYSIS

A. Why did fine-tuning hurt?

Shorter reasoning. The median C4 answer has 84 tokens against 211 for C1. GSM8K reference solutions are terse, and C4 adopted that style completely: 55% of C1 answers use LaTeX-formatted step-by-step derivations, against 0% of C4 answers.

The loss concentrates on long problems. We bucket test items by the number of steps in their reference solution (Fig. 3). The C1−C4 gap grows almost monotonically with problem length: 6.7 points for 1–2 steps, 8.6 for 3, 10.1 for 4, 19.4 for 5 and 20.7 for 6 or more. A model that writes less reasoning fails where more reasoning is needed.

Arithmetic is not the cause. We re-evaluated every equation of the form "*expression* = *number*" found in the outputs, respecting operator precedence. The error rate per equation is similar for C1 (3.8%), C2 (3.1%) and C4 (3.5%). The sampled C4 failures are planning errors with correct arithmetic: dropped constraints, wrong quantities combined, or misread units.

Interpretation. Qwen2.5-Instruct was already post-trained extensively on mathematical reasoning, and its own derivations are longer and more careful than the human references. Supervised fine-tuning on those references therefore imitates a weaker teacher. The low initial loss (about 0.6) and the early plateau suggest more steps would not help. The standard remedy is to train on the model's own verified solutions instead (rejection-sampling fine-tuning, STaR [7], [8]).

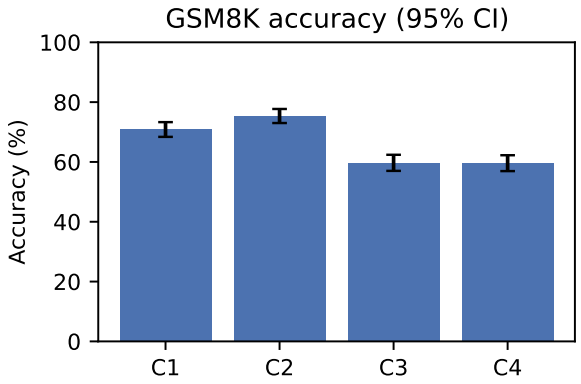


Fig. 1. Accuracy per condition with 95% bootstrap CIs.

TABLE III
COST OF EACH APPROACH.

	Prompting (C1–C3)	LoRA (C4)
Trainable parameters	0	18.46M (1.18%)
Training time (A100)	0	9.2 min
Peak GPU memory, training	–	7.1 GB
Mean generated tokens	217 / 278 / 110	95
Latency per item, batch 128	0.17 / 0.18 / 0.09 s	0.27 s

B. Why did PAL not help?

PAL removes arithmetic errors, but at about 3% per equation there are few to remove. Its costs are larger: 7.4% of programs gave no answer (85 ended with an assignment instead of the requested `solution()` function; 12 raised errors, none timed out), and only 64.4% of the executable programs were correct, because translating a word problem into code is itself a reasoning step at which a 1.5B model fails. As a sensitivity check that is *not* our primary metric, reading the last assigned variable of those 85 programs raises PAL to 64.1%, still 6.8 points below C1.

C. Direct prompting is already implicit CoT

With a mean of 217 tokens, the "direct" C1 condition is not answer-only: the instruction-tuned model reasons step by step without being asked. The explicit CoT instruction adds structure and verification on top of this, which explains a moderate rather than dramatic gain.

D. Complementarity

Conditions fail on different items: 88.2% of problems are solved by at least one condition, but only 38.1% by all four. CoT and PAL return the same answer on 56.8% of items, and on those items that answer is correct 93.6% of the time; on the remaining items CoT accuracy falls to 51.4%. Agreement between two cheap prompting strategies is therefore a useful confidence signal for deciding when to trust an answer.

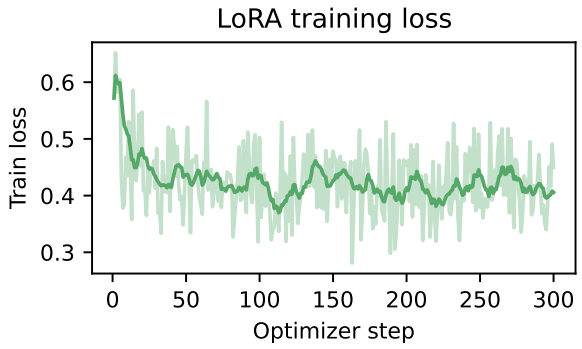


Fig. 2. LoRA training loss (raw and 10-step moving average).

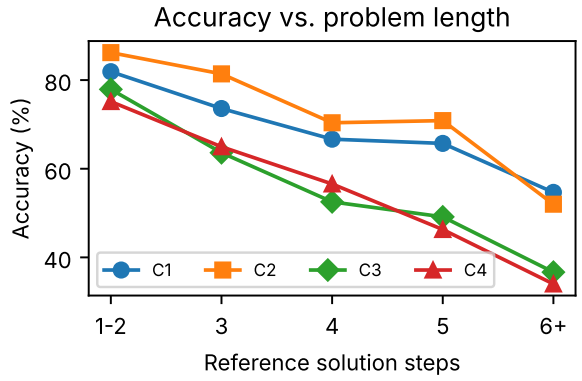


Fig. 3. Accuracy by number of steps in the reference solution. Bucket sizes: 326, 371, 297, 175, 150.

V. Discussion

The cost comparison favours prompting. C2 needs no training and adds only 61 tokens per answer over C1, while C4 needed a training run and still lost. C4 is also slower per item despite shorter outputs. A likely cause is that its LoRA adapters were not merged into the base weights, adding extra matrix products in every layer; merging would remove this overhead. More generally, fine-tuning helps when the supervision is better than what the model already does. On a saturated benchmark with a strong instruction-tuned model, that is not the case for human references, and the result reverses the common intuition that fine-tuning always beats prompting.

VI. Limitations

We use one model, one seed and one LoRA configuration, so variance across seeds and the effect of rank, learning rate and data are not measured. GSM8K may overlap the model's post-training data; this affects all conditions but may reduce the headroom for fine-tuning. Greedy decoding excludes self-consistency, which typically improves CoT. The arithmetic checker only parses explicit numeric equations, so its error rates describe the checkable subset of each output. Latency is measured per batch and depends on batch composition.

VII. Conclusion

No: a single LoRA run on GSM8K reference solutions did not outperform prompting. It lost 11.3 points to direct prompting and 15.8 points to zero-shot CoT on the same model, both highly significant. The failure traces to the training data, not the method. Fine-tuning taught the model to reason more briefly, which hurts most on long problems. For this model and task, zero-shot chain-of-thought is the best of the tested options.

Reproducibility

Code, Colab notebook, raw per-item predictions and the analysis script are available at https://github.com/MeteHanYilmaz0/Data_Ai/tree/ccr-36b08569-obh9ku/YZM5109_Assignment1.

Use of AI Tools

An AI assistant (Claude) was used to design and implement the experiment code, to carry out the post-hoc analysis and to draft this report. The author ran all experiments on Google Colab and reviewed the code, the results and the text.

References

[1] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
[2] Qwen Team, "Qwen2.5 technical report," *arXiv:2412.15115*, 2024.
[3] K. Cobbe *et al.*, "Training verifiers to solve math word problems," *arXiv:2110.14168*, 2021.
[4] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022.
[5] T. Kojima *et al.*, "Large language models are zero-shot reasoners," in *Proc. NeurIPS*, 2022.
[6] L. Gao *et al.*, "PAL: Program-aided language models," in *Proc. ICML*, 2023.
[7] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in *Proc. NeurIPS*, 2022.
[8] Z. Yuan *et al.*, "Scaling relationship on learning mathematical reasoning with large language models," *arXiv:2308.01825*, 2023.
[9] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.